Chapter 3: Calculus and Optimization Essentials for Machine Learning

Dr. Youssef SALMAN

Applied Mathematics
Lebanese University – Faculty of Sciences (Section V)

M1: Applied Mathematics & Statistics for Computational Sciences

September 21, 2026

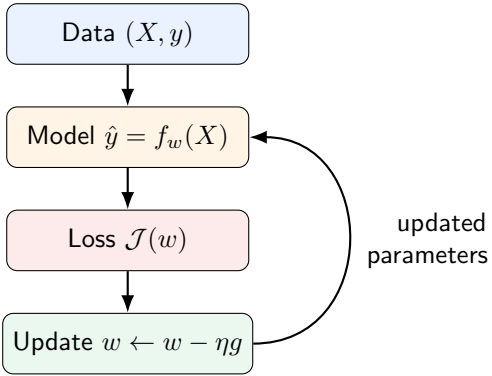
Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

Why this chapter?

- A model is not useful until we choose its parameters.
- Training is an optimization problem: reduce a loss on data.
- Calculus tells us where and how fast the loss changes.
- Optimization turns this information into update rules.

How this serves ML: Optimization is the engine behind linear regression, logistic regression, neural networks, regularization, and constrained learning.



Variable dictionary for the whole chapter

We use the following notation before writing any algorithm.

Symbol	Meaning
n	number of training observations
p	number of features / parameters
$X \in \mathbb{R}^{n \times p}$	design matrix: row i is $x_i^\top$
$y \in \mathbb{R}^n$	target vector, with entry y_i
$w \in \mathbb{R}^p$	vector of model parameters to learn
$\hat{y}_i = f_w(x_i)$	model prediction for observation i
$\ell_i(w)$	loss contributed by observation i
$\mathcal{J}(w) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(w) + \lambda R(w)$	objective minimized during training
$\eta > 0$	learning rate / step size

How this serves ML: This dictionary avoids ambiguity: every gradient descent variant later differs only in how it estimates $\nabla \mathcal{J}(w)$.

Roadmap of the chapter

1. Single-variable recap: limit, continuity, derivative.
2. Multivariate calculus: gradient, Jacobian, Hessian, Taylor expansion.
3. Smoothness and convexity: when steps are safe and minima are reliable.
4. Optimization algorithms: full-batch GD, SGD, mini-batch GD, Newton, projection.
5. Python implementation: clear variables, then runnable code.

How this serves ML: The objective is not theory for theory's sake: every slide answers "how does this help train an ML model?"

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

Limit, continuity, derivative: the basic language

Let $f : \mathbb{R} \to \mathbb{R}$ and let $a \in \mathbb{R}$.

Definitions

- Limit: $\lim_{x \to a} f(x) = L$ means $f(x)$ approaches L when x approaches a.
- Continuity at a: $\lim_{x \to a} f(x) = f(a)$.
- Derivative at a: $f'(a) = \lim_{h \to 0} \dfrac{f(a+h) - f(a)}{h}$.

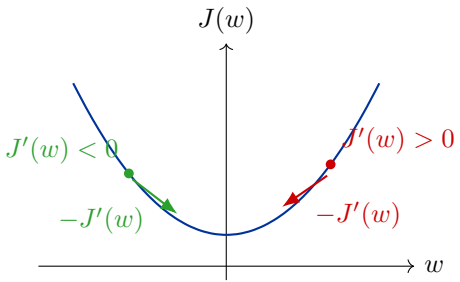
How this serves ML: A derivative measures how the loss changes when one parameter is slightly modified. This is the local signal used to update parameters.

Derivative sign: which direction reduces the loss?

For a one-dimensional loss $J(w)$:

- if $J'(w) > 0$, increasing w increases the loss;
- to reduce the loss, move left: $w \leftarrow w - \eta J'(w)$;
- if $J'(w) < 0$, the same formula moves right.

How this serves ML: Gradient descent is just this idea generalized to many parameters.



Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

From one parameter to many parameters

Let $w = (w_1, \ldots, w_p)^\top \in \mathbb{R}^p$ be the vector of parameters.

Gradient

For a scalar objective $\mathcal{J} : \mathbb{R}^p \to \mathbb{R}$,

$$\nabla \mathcal{J}(w) = \begin{bmatrix} \frac{\partial \mathcal{J}}{\partial w_1}(w) \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_p}(w) \end{bmatrix} \in \mathbb{R}^p.$$

- $\nabla \mathcal{J}(w)$ points toward the steepest increase.
- $-\nabla \mathcal{J}(w)$ points toward the steepest local decrease.

How this serves ML: For ML, w may contain all weights of a model; the gradient tells us how to update all weights simultaneously.

Directional derivative: testing a proposed move

Let $u \in \mathbb{R}^p$ be a direction with $\|u\| = 1$.

Definition

$$D_u \mathcal{J}(w) = \lim_{h \to 0} \frac{\mathcal{J}(w + hu) - \mathcal{J}(w)}{h} = \nabla \mathcal{J}(w)^\top u.$$

- If $D_u \mathcal{J}(w) < 0$, then moving a little in direction u decreases the loss.
- The most decreasing direction is $u = -\nabla \mathcal{J}(w) / \|\nabla \mathcal{J}(w)\|$.

How this serves ML: This explains why the negative gradient is used in training: it is the locally best descent direction.

Jacobian: derivatives of vector-valued functions

Let $g : \mathbb{R}^p \to \mathbb{R}^m$ with $g(w) = \big(g_1(w), \ldots, g_m(w)\big)^\top$.

Jacobian matrix

$$J_g(w) = \begin{bmatrix} \frac{\partial g_1}{\partial w_1} & \cdots & \frac{\partial g_1}{\partial w_p} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial w_1} & \cdots & \frac{\partial g_m}{\partial w_p} \end{bmatrix} \in \mathbb{R}^{m \times p}.$$

How this serves ML: In neural networks, each layer is vector-valued. Backpropagation repeatedly multiplies by Jacobians through the chain rule.

Hessian: curvature of the loss

Let $\mathcal{J} : \mathbb{R}^p \to \mathbb{R}$ be twice differentiable.

Hessian matrix

$$H_{\mathcal{J}}(w) = \nabla^2 \mathcal{J}(w) = \left[\frac{\partial^2 \mathcal{J}}{\partial w_i \partial w_j}(w) \right]_{i,j=1}^{p} \in \mathbb{R}^{p \times p}.$$

- Gradient = slope information.
- Hessian = curvature information.
- Positive curvature near a critical point indicates a local minimum.

How this serves ML: Second-order methods such as Newton use curvature to adapt the direction and scale of the step.

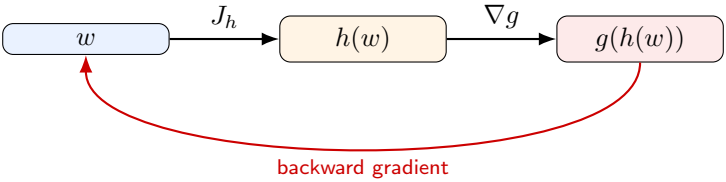
Chain rule: the mathematical core of backpropagation

Let $h : \mathbb{R}^p \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}$. Define

$$\mathcal{J}(w) = g(h(w)).$$

Vector chain rule

$$\nabla \mathcal{J}(w) = J_h(w)^\top \nabla g(h(w)).$$



How this serves ML: Backpropagation is an efficient repeated application of this rule from the output loss back to every parameter.

Taylor approximation: why local models work

Let $s \in \mathbb{R}^p$ be a small proposed step from w.

First- and second-order approximations

$$\mathcal{J}(w+s) \approx \mathcal{J}(w) + \nabla \mathcal{J}(w)^\top s,$$

$$\mathcal{J}(w+s) \approx \mathcal{J}(w) + \nabla \mathcal{J}(w)^\top s + \frac{1}{2} s^\top H_{\mathcal{J}}(w) s.$$

- Gradient descent uses the **first-order approximation**.
- Newton's method uses the **second-order approximation**.

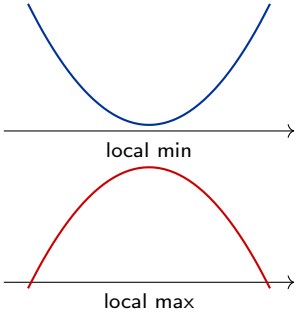
How this serves ML: Optimization algorithms do not see the whole loss landscape; they use local approximations to decide how the model parameters should move at the next iteration.

Critical points: minimum, maximum, saddle

A point $w^\star$ is critical when $\nabla \mathcal{J}(w^\star) = 0$.

- If $H_{\mathcal{J}}(w^\star) \succ 0$, it is locally bowl-shaped: local minimum.
- If $H_{\mathcal{J}}(w^\star) \prec 0$, it is locally cap-shaped: local maximum.
- If the Hessian is indefinite, it is a saddle point.

How this serves ML: Deep learning often has many saddle points; optimization must keep moving when zero or small gradients appear.



Smoothness: a condition for safe learning rates

A differentiable function $\mathcal{J}$ is called L-smooth if

$$\|\nabla \mathcal{J}(u) - \nabla \mathcal{J}(v)\| \leq L\|u - v\| \quad \text{for all } u, v.$$

- L controls how fast the gradient can change.
- Large L means strong curvature: steps must be smaller.
- For many convex quadratic losses, $\eta \leq 1/L$ is a safe choice.

How this serves ML: Smoothness helps choose a learning rate that reduces loss instead of overshooting.

Convexity: when local minimum means global minimum

A function $\mathcal{J} : \mathbb{R}^p \to \mathbb{R}$ is convex if

$$\mathcal{J}(\theta u + (1-\theta)v) \leq \theta \mathcal{J}(u) + (1-\theta)\mathcal{J}(v), \quad 0 \leq \theta \leq 1.$$

If $\mathcal{J}$ is twice differentiable, convexity is equivalent to

$$H_{\mathcal{J}}(w) \succeq 0 \quad \text{for all } w.$$

- Convex problems have no bad local minima.
- Strong convexity often gives faster and more stable convergence.

How this serves ML: Linear regression, ridge regression, logistic regression, and many classical ML objectives are convex.

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

Least squares: all variables initialized

Given $X \in \mathbb{R}^{n \times p}$, $y \in \mathbb{R}^n$, and parameters $w \in \mathbb{R}^p$, define predictions

$$\hat{y} = Xw.$$

The mean squared error objective is

$$\mathcal{J}(w) = \frac{1}{2n}\|Xw - y\|^2.$$

Gradient and Hessian

$$\nabla \mathcal{J}(w) = \frac{1}{n} X^\top (Xw - y), \qquad H_{\mathcal{J}}(w) = \frac{1}{n} X^\top X.$$

How this serves ML: This is the cleanest example where optimization trains a model: find the weights w that make Xw close to y.

Logistic regression: all variables initialized

For binary classification, $y_i \in \{0, 1\}$ and $x_i \in \mathbb{R}^p$.

$$z_i = x_i^\top w, \qquad p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

The average negative log-likelihood is

$$\mathcal{J}(w) = \frac{1}{n} \sum_{i=1}^{n} \left[-y_i \log(p_i) - (1 - y_i) \log(1 - p_i) \right].$$

Gradient

Let $p = (p_1, \ldots, p_n)^\top$. Then

$$\nabla \mathcal{J}(w) = \frac{1}{n} X^\top (p - y).$$

How this serves ML: Optimization adjusts w so predicted probabilities become closer to the class labels.

Worked function used later

Let $x = (x_1, x_2)^\top \in \mathbb{R}^2$ and define

$$f(x_1, x_2) = x_1^2 + 2x_2^2 + e^{-x_1 - x_2}.$$

Then

$$\nabla f(x) = \begin{bmatrix} 2x_1 - e^{-x_1 - x_2} \\ 4x_2 - e^{-x_1 - x_2} \end{bmatrix},$$

$$H_f(x) = \begin{bmatrix} 2 + e^{-x_1 - x_2} & e^{-x_1 - x_2} \\ e^{-x_1 - x_2} & 4 + e^{-x_1 - x_2} \end{bmatrix} \succ 0.$$

How this serves ML: Because the Hessian is positive definite, the function is convex and has a unique minimizer. This is ideal for demonstrating optimization behavior.

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

What is optimization in ML?

A standard supervised learning problem is

$$\min_{w \in \mathbb{R}^p} \; \mathcal{J}(w) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_w(x_i), y_i\big) + \lambda R(w).$$

- ℓ measures prediction error.
- $R(w)$ is a regularizer that controls model complexity.
- $\lambda \geq 0$ controls the strength of regularization.

How this serves ML: Optimization is how learning happens: model architecture defines f_w, data define the loss, optimization finds useful w.

Generic iterative algorithm

Start from an initial parameter vector $w_0 \in \mathbb{R}^p$.

$$w_{k+1} = w_k + \text{step}_k, \qquad k = 0, 1, 2, \ldots$$

For gradient methods,

$$w_{k+1} = w_k - \eta_k g_k.$$

- g_k estimates the gradient at w_k.
- $\eta_k > 0$ controls how far we move.
- The objective is to reduce $\mathcal{J}(w_k)$ over iterations.

How this serves ML: Most ML optimizers are variations of this update: change how g_k is computed and how η_k is chosen.

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

One update formula, three choices of gradient

Assume

$$\mathcal{J}(w) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(w), \quad \text{where } \ell_i(w) = \ell(f_w(x_i), y_i).$$

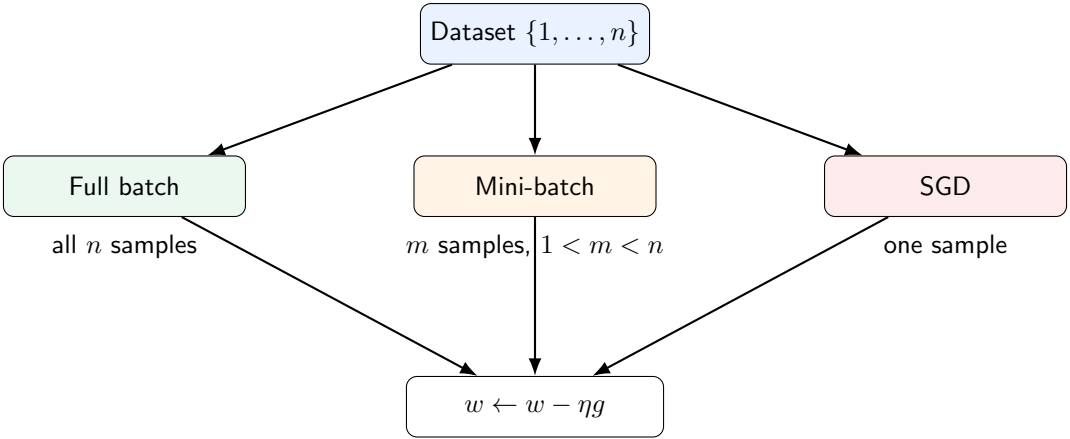
At iteration k:

$$w_{k+1} = w_k - \eta g_k.$$

Method	Gradient used g_k	Data used per update
Full-batch GD	$\frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i(w_k)$	all n observations
SGD	$\nabla \ell_{i_k}(w_k)$	one observation
Mini-batch GD	$\frac{1}{m}\sum_{i \in B_k} \nabla \ell_i(w_k)$	m observations

How this serves ML: The three methods differ by a speed-versus-noise tradeoff. This is central in training large ML models.

TikZ view: full-batch vs SGD vs mini-batch



How this serves ML: Full-batch is stable but expensive; SGD is cheap but noisy; mini-batch is the practical default in neural networks.

Full-batch gradient descent: clear interpretation

At iteration k, compute the exact empirical gradient:

$$g_k = \nabla \mathcal{J}(w_k) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(w_k).$$

Then update:

$$w_{k+1} = w_k - \eta g_k.$$

- Uses all observations before making one update.
- Low noise in direction.
- Can be slow when n is very large.

How this serves ML: Useful for small or medium data, convex models, and when stable convergence is more important than speed.

SGD: clear interpretation

Choose one random index $i_k \in \{1, \dots, n\}$ and compute

$$g_k = \nabla \ell_{i_k}(w_k).$$

Then update:

$$w_{k+1} = w_k - \eta g_k.$$

- Very cheap update.
- Noisy direction.
- Noise can help escape shallow regions, but can also cause oscillations.

How this serves ML: SGD made large-scale ML possible because it does not require scanning the whole dataset at every step.

Mini-batch GD: clear interpretation

Choose a random mini-batch $B_k \subset \{1, \dots, n\}$ with size $|B_k| = m$.

$$g_k = \frac{1}{m} \sum_{i \in B_k} \nabla \ell_i(w_k).$$

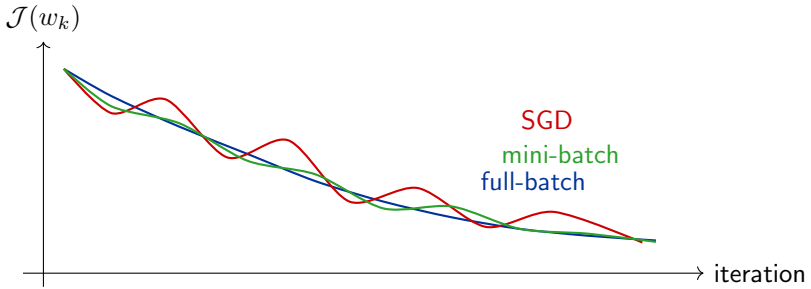
Then update:

$$w_{k+1} = w_k - \eta g_k.$$

- $m = 1$: SGD.
- $m = n$: full-batch GD.
- $1 < m < n$: compromise between noise and cost.

How this serves ML: Mini-batches use hardware efficiently and reduce gradient noise. This is the default in deep learning.

Noise comparison on the same objective



How this serves ML: Batch size controls the smoothness of the training trajectory: larger batches are smoother but each update costs more.

Python: full-batch GD on $f(x) = x^2$

```python
import numpy as np

# Objective: f(x) = x^2, variable x in R
# Exact gradient: f'(x) = 2x

def f(x):
    return x**2

def grad_f(x):
    return 2*x

x = 50.0 # initial value x_0
eta = 0.01 # learning rate
n_steps = 1000 # number of updates
path = [x]

for k in range(n_steps):
    g = grad_f(x) # exact/full gradient
    x = x - eta*g # gradient descent update
    path.append(x)

print("final x =", x, "final f(x) =", f(x))
```

Python: SGD on a simple stochastic objective

We want to minimize $f(x) = x^2$. We use a noisy unbiased gradient

$$g = 2(x - Z), \qquad Z \sim \mathcal{N}(0, 1), \qquad \mathbb{E}[g \mid x] = 2x.$$

```python
import numpy as np

rng = np.random.default_rng(0)

def f(x):
    return x**2

x = 80.0
eta = 0.01
n_steps = 1000
path = [x]

for k in range(n_steps):
    z = rng.normal(0.0, 1.0) # one random sample
    g = 2*(x - z) # noisy but unbiased gradient
    x = x - eta*g
    path.append(x)

print("final x =", x, "final f(x) =", f(x))
```

Python: mini-batch GD on the same objective

Use the average of m noisy samples: $g = 2\left(x - \frac{1}{m}\sum_{j=1}^{m} Z_j\right)$.

```python
import numpy as np

rng = np.random.default_rng(0)

def f(x):
    return x**2

x = 80.0
eta = 0.01
n_steps = 600
m = 32 # mini-batch size
path = [x]

for k in range(n_steps):
    z_batch = rng.normal(0.0, 1.0, size=m)
    g = 2*(x - z_batch.mean())
    x = x - eta*g
    path.append(x)

print("final x =", x, "final f(x) =", f(x))
```

Practical batch-size summary

Method	Advantage	Limitation	Typical use
Full-batch	accurate direction	expensive for huge data	small/medium datasets
SGD	very cheap updates	high variance, oscillations	online learning, huge data
Mini-batch	balanced cost/noise; hardware friendly	batch size must be tuned	neural networks, modern ML

How this serves ML: In practice, start with mini-batch GD, tune η, then tune batch size m according to hardware and validation performance.

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

Newton's method: curvature-aware optimization

At iteration k, let

$$g_k = \nabla \mathcal{J}(w_k), \qquad H_k = H_{\mathcal{J}}(w_k).$$

Newton chooses a step s_k by solving

$$H_k s_k = g_k,$$

and updates

$$w_{k+1} = w_k - s_k.$$

- It uses gradient and Hessian.
- It can converge very fast near a well-conditioned minimum.
- It is expensive when p is large because solving a linear system costs more than a gradient step.

How this serves ML: Newton is powerful for small or medium models; for large neural networks, approximations are usually needed.

Newton step from Taylor's formula

Use the quadratic approximation

$$\mathcal{J}(w+s) \approx \mathcal{J}(w) + g^\top s + \frac{1}{2} s^\top H s.$$

Differentiate with respect to s:

$$\nabla_s \left(g^\top s + \frac{1}{2} s^\top H s \right) = g + Hs.$$

Set this equal to zero:

$$Hs = -g \quad \Longrightarrow \quad s = -H^{-1} g.$$

So the update is

$$w_{k+1} = w_k - H_k^{-1} g_k.$$

How this serves ML: Gradient descent trusts only slope; Newton also asks how curved the loss is before moving.

Python: one Newton update for logistic regression

```python
import numpy as np

def sigmoid(z):
    return 1.0/(1.0 + np.exp(-z))

# X: n x p design matrix
# y: length-n vector with entries 0 or 1
# w: length-p parameter vector
# lam: L2 regularization strength

def logreg_grad(X, y, w, lam=0.0):
    n = len(y)
    p_hat = sigmoid(X @ w)
    return (X.T @ (p_hat - y))/n + lam*w

def logreg_hess(X, y, w, lam=0.0):
    n, p = X.shape
    p_hat = sigmoid(X @ w)
    W = p_hat*(1 - p_hat) # length n
    H = (X.T * W) @ X / n + lam*np.eye(p)
    return H

def newton_update(X, y, w, lam=0.0):
    g = logreg_grad(X, y, w, lam)
    H = logreg_hess(X, y, w, lam)
    step = np.linalg.solve(H, g) # solve H step = g
    return w - step
```

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

Convex sets and constrained learning

A set $C \subset \mathbb{R}^p$ is convex if for every $u, v \in C$ and $0 \leq \theta \leq 1$,

$$\theta u + (1 - \theta)v \in C.$$

A constrained ML problem has the form

$$\min_{w \in C} \mathcal{J}(w).$$

Examples:

- box constraints: $a_j \leq w_j \leq b_j$;
- nonnegative weights: $w_j \geq 0$;
- norm constraints: $\|w\|_2 \leq r$.

How this serves ML: Constraints encode domain rules, numerical stability, sparsity, fairness restrictions, or bounded coefficients.

Projection idea

For a closed convex set C, projection means

$$\mathrm{proj}_C(z) = \arg\min_{u \in C} \|u - z\|_2.$$

Projected gradient descent is

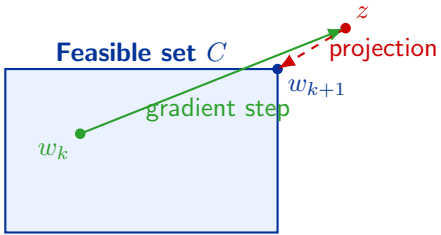
$$w_{k+1} = \mathrm{proj}_C\left(w_k - \eta \nabla \mathcal{J}(w_k)\right).$$

For box constraints $C = [a_1, b_1] \times \cdots \times [a_p, b_p]$,

$$\left(\mathrm{proj}_C(z)\right)_j = \min\{\max\{z_j, a_j\}, b_j\}.$$

How this serves ML: Projection lets us train while forcing parameters to remain feasible after every update.

TikZ view: gradient step then projection



$$z = w_k - \eta \nabla \mathcal{J}(w_k), \qquad w_{k+1} = \mathrm{proj}_C(z).$$

How this serves ML: A raw gradient step may leave the allowed region; projection brings it back to the closest feasible point.

Python: projected GD on a convex quadratic

```python
import numpy as np

# Minimize f(x) = 0.5 x^T Q x + c^T x
# subject to a <= x <= b, coordinate-wise

Q = np.array([[3.0, 1.0],
              [1.0, 2.0]]) # positive definite matrix
c = np.array([-2.0, 1.0])

def f(x):
    return 0.5*x.T @ Q @ x + c.T @ x

def grad_f(x):
    return Q @ x + c

a = np.array([-2.0, -2.0]) # lower bounds
b = np.array([ 2.0, 2.0]) # upper bounds

def proj_box(z):
    return np.minimum(np.maximum(z, a), b)

L = np.max(np.linalg.eigvals(Q).real)
eta = 1.0/L
x = np.array([1.8, -1.5])
path = [x.copy()]

for k in range(100):
    x = proj_box(x - eta*grad_f(x))
    path.append(x.copy())
```

Roadmap

1. Motivation and notation
2. Single-variable recap
3. Multivariate calculus essentials
4. Core ML derivatives
5. Optimization basics
6. Gradient descent family
7. Newton's method
8. Constrained and convex optimization
9. Takeaways

What students should remember

- Calculus gives local information: derivative, gradient, Hessian.
- Optimization converts this information into parameter updates.
- Full-batch, SGD, and mini-batch use the same update but different gradient estimates.
- Smoothness helps choose safe learning rates.
- Convexity makes optimization easier; non-convex ML needs practical heuristics.
- Projection handles constraints by correcting infeasible updates.

How this serves ML: The bridge is: data $\rightarrow$ loss $\rightarrow$ gradient $\rightarrow$ update $\rightarrow$ better model.

Suggested lab after this lecture

Students can modify three quantities in Python:

1. learning rate η;
2. number of iterations n_steps;
3. mini-batch size m.

They should observe:

- too small η: slow training;
- too large η: oscillation or divergence;
- larger m: smoother but more expensive updates.

How this serves ML: This turns optimization from formulas into visible ML behavior.

Thank you!

Questions?

